Chapter 9
Accessing Map Layers

If you have accomplished Exercise 2 of Chapter 7, you should have created an add-in project named “CityGIS”. Otherwise, you can find the project in folder “VB2010” on the CDROM and copy it to your hard drive. This project includes a toolbar that contains one button, two tools and one combo box. The button is intended for turning on and off all map layers in ArcMap at one click, the tools will be used to create bus stops and routes (point and line elements), and the combo box will allow the user to select a map scale from a list to zoom to. Functionality of these add-in components are not implemented yet. Starting from this chapter, you will gradually write code to implement these components. As you progress with the book, you will also create more add-in components in the CityGIS toolbar and implement more GIS functions.

This chapter will first introduce two key components, Application and MxDocument, which are usually entry points into the ArcObjects world. Then it will show the Map and Layer components in the Carto assembly and demonstrates how to manipulate maps and layers in a map document. To practice programming with ArcObjects, this chapter will guide you to implement the add-in components in the CityGIS project. Specifically, you will implement the turn on/off layers button and the map scale combo box, and you will partially implement the bus stop tool so that it reports coordinates of mouse clicking locations. In addition, you will learn a few add-in project related techniques including loading assemblies into an add-in project, importing namespaces into code module, presenting information on the status bar, as well as setting tool cursors.

1. Application and MxDocument

ArcObjects provides components for developers to use code to manipulate map layers, features in layers, attributes of features, as well as to call ArcGIS built-in functions and tools. Please recall that ArcObjects components are organized in assemblies (i.e. packages) each having a unique namespace. When you create a new ArcGIS Desktop add-in project from a template, the following ESRI assemblies are loaded into your project by the wizard:

- ESRI.ArcGIS.Framework – This assembly includes the Application object with IApplication interface and the Document object with IDocument interface.
- ESRI.ArcGIS.ArcMapUI – This assembly includes the MxDocument object with IMxDocument interface.
- ESRI.ArcGIS.Desktop.AddIns – This assembly includes add-in foundation classes.
- ESRI.ArcGIS.System – this assembly includes ArcGIS system classes.

You can find these ESRI assemblies along with a number of Microsoft assemblies in an ArcGIS add-in project by expanding the “References” node in the Solution Explorer. If the “References” node does not show in the Solution Explorer, click the “Show All Files” button to display it (Figure 1).

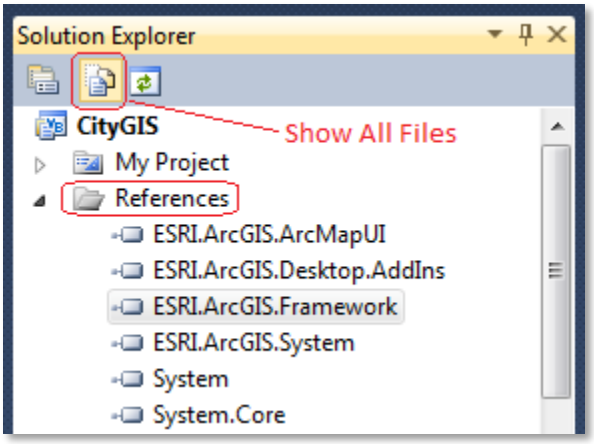


Figure 1 References in an ArcGIS add-in project

With the Framework assembly and ArcMapUI assembly preloaded in an add-in project, two key objects, Application and MxDocument, are automatically instantiated and ready for use. If we compare the entire ArcObjects library to a city, the Application and MxDocument objects are two airports where we can land. These airports can serve as starting points from which we explore the fantastic city.

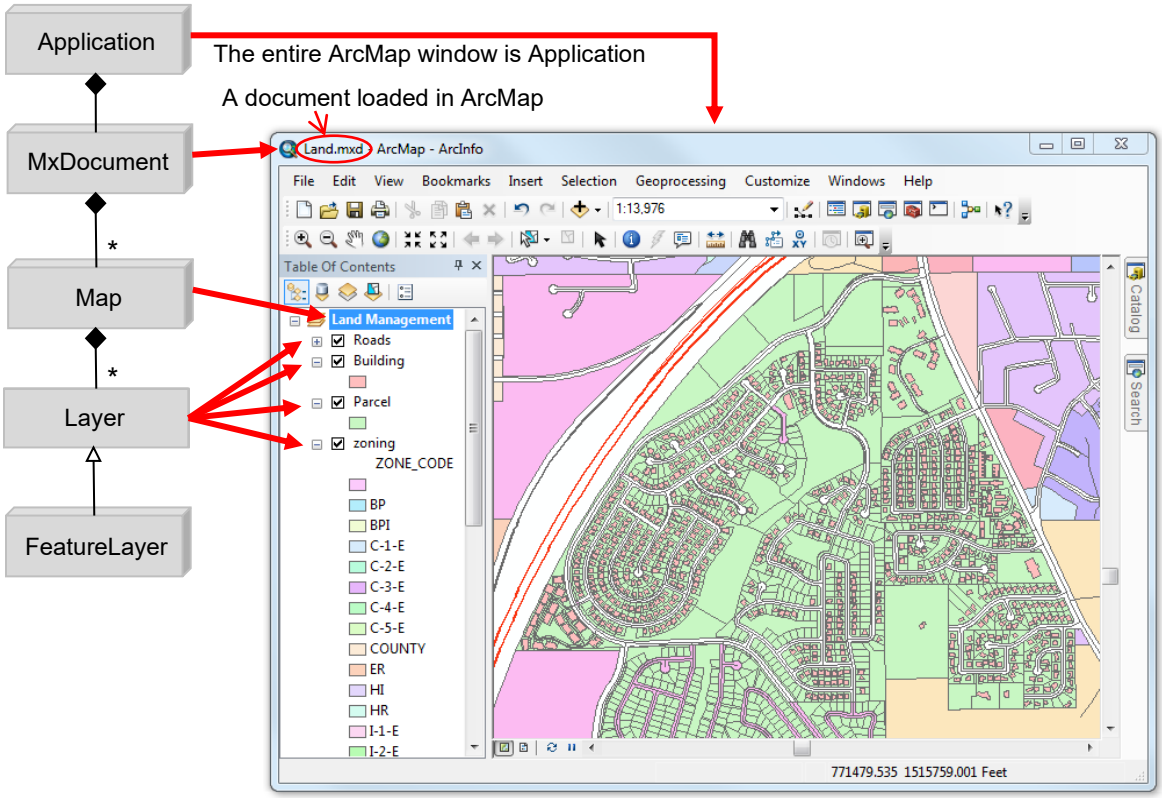


Figure 2 Components of an ArcMap application

In an add-in project for ArcMap, the Application object refers to the ArcMap application which is represented by the entire ArcMap window (Figure 2). This object provides interfaces for the user to control the ArcMap window. With the Application object, you can change the caption of

ArcMap, you can display information on its status bar (at the bottom), you can save a map document loaded in the application, and you can shut down ArcMap. The Application object has several interfaces among which the default interface is IApplication. You can find useful properties such as Caption and StatusBar, as well as useful methods including SaveDocument(), PrintDocument(), PrintPreview(), and Shutdown() on the IApplication interface (Figure 3). Please open the full object model diagram "FrameworkObjectModel.pdf" (available on the CD-ROM) to look at other interfaces of the Application object.

To use the Application object in a program, we usually declare a variable to point to an interface of the object, and then assign the object to the variable. Technically, this is to store the address of the Application object in that variable. By doing so, the variable becomes a proxy of the Application object with the specified interface. For example,

Assigns the Application object to variable app

```
Dim app As ESRI.ArcGIS.Framework.IApplication = My.ArcMap.Application
```

Declares a variable to point to the IApplication interface

With the IApplication interface, we can use set or change the caption of the ArcMap window. This is accomplished by setting a value to the Caption property on IApplication interface. For example,

```
app.Caption = "Flagstaff City GIS"
```

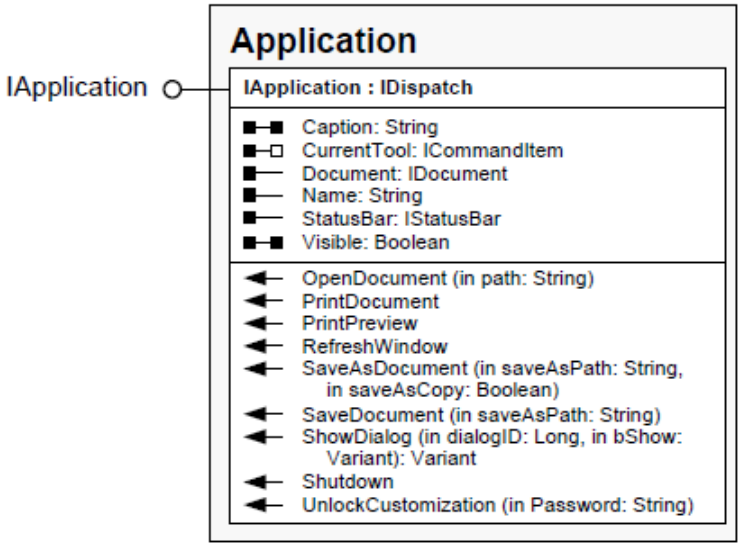


Figure 3 Part of the Application component

A map document in ArcGIS is saved as a binary file with ".mxd" extension. One ArcMap application can only contains one map document. With ArcObjects, the MxDocument object represents the map document loaded in an ArcMap application. A map document can contain multiple maps. In ArcMap, a map is also referred to as a Data Frame. The default map in a document is automatically labeled with "Layers" in the Table of Contents (TOC) panel in

ArcMap. With the MxDocument object, you can access the currently active map object (i.e. the active data frame in ArcMap) through the FocusMap property on the IMxDocument interface. You can also add layers into a map by calling the AddLayer() method also on the IMxDocument interface (Figure 4).

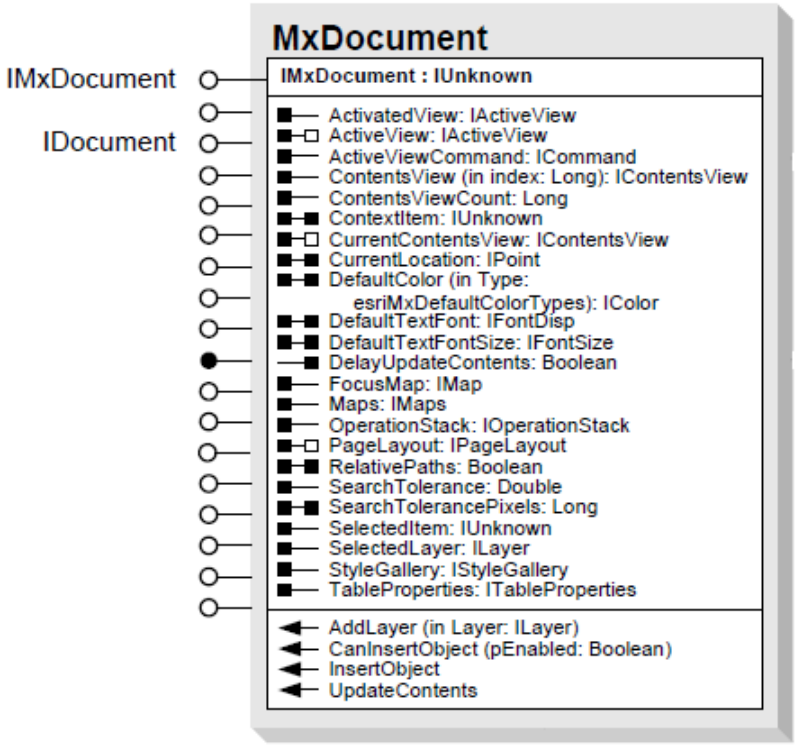


Figure 4 Part of the MxDocument component

Please open the "ArcMapUIObjectModel.pdf" document, zoom into the MxDocument class and explore the class a little bit to find more interfaces and their properties and methods. In Figure 4, you should find the FocusMap property, the Maps property, and the AddLayer() method all exist on the IMxDocument interface which is the default interface of the MxDocument class. Like the Application object, you can use a proxy variable to represent the MxDocument object for convenience of use. For example,

```
Dim mxDoc As ESRI.ArcGIS.ArcMapUI.IMxDocument = My.ArcMap.Document
mxDoc.AddLayer(<a layer object>)      ' to call AddLayer method of the object
```

To find more information about the diagram to the left of the ArcMap screenshot in Figure 2, please recall that in UML a line with a diamond on one end indicates an aggregation (whole-part) relationship. And an asterisk symbol in an aggregation relationship denotes multiplicity. Figure 2 indicates that the MxDocument object can contain multiple Map objects, and a Map object can contain multiple Layer objects. Moreover, the Layer class is an abstract class as indicated by a 2-D gray box, and it is inherited by the FeatureLayer coclass as well as other coclasses including RasterLayer and TINLayer which are not shown in the diagram.

2. The Carto Assembly

The Application and MxDocument objects do not directly operate map layers in a map document. To turn on or off a map layer, you must get the Map object from the MxDocument object, and then get the Layer object from the Map object. The Layer class has a property named "Visible" which allows you to turn the display of a layer on or off. The Map and Layer objects are included in an assembly named Carto. To use these objects, you must add this assembly into your project as a reference.

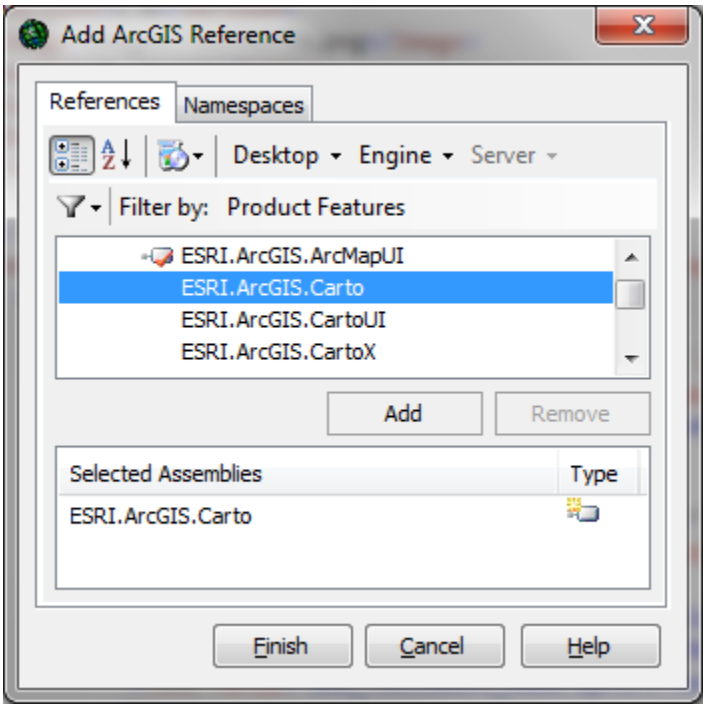


Figure 5 Adding .ArcGIS assemblies into project as references

To add the Carto assembly into your CityGIS project,

1) In the Solution Explorer, right click the "References", and then select "Add ArcGIS References…"
2) In the Add ArcGIS Reference dialog box, scroll up and down to find "ESRI.ArcGIS.Carto" item, highlight it and click Add (Figure 5). The item should be added into the lower box. If necessary, you can add more assemblies into the lower box, and click the "Finish" button to finish.
3) Expand the Reference node in the Solution Explorer to check if the assembly is added

Since almost every ArcGIS desktop add-in project deals with map layers, you are suggested to add the Carto assembly as soon as you create a new ArcGIS add-in project. After adding the Carto assembly into the project, you can get the focus map, i.e. the active data frame, from the MxDocument object (IMxDocument interface) and use a variable to represent it, for example,

```
Dim map As ESRI.ArcGIS.Carto.IMap = mxDoc.FocusMap
```

In the above code, we obtain a map object from the MxDocument object which is represented by variable *mxDoc*. This was achieved by calling the FocusMap property on the IMxDocument

interface of the MxDocument object. The object model diagram in Figure 6 shows that the FocusMap property returns an IMap interface, i.e. a map object with IMap interface. To preserve the Map object returned from the FocusMap property for later use, a variable named *map* is declared to point to the IMap interface. Declaring a variable to point to an interface of an ArcObjects component is the same as declaring a variable of a simple data type such as String or Integer except that an interface is usually referenced with a long namespace. In the example code line above, the IMap interface is referenced with a namespace ESRI.ArcGIS.Carto. Now, variable *map* is a proxy of the current (active) map in the ArcMap document. Moreover, variable *map* points to the IMap interface of the map object.

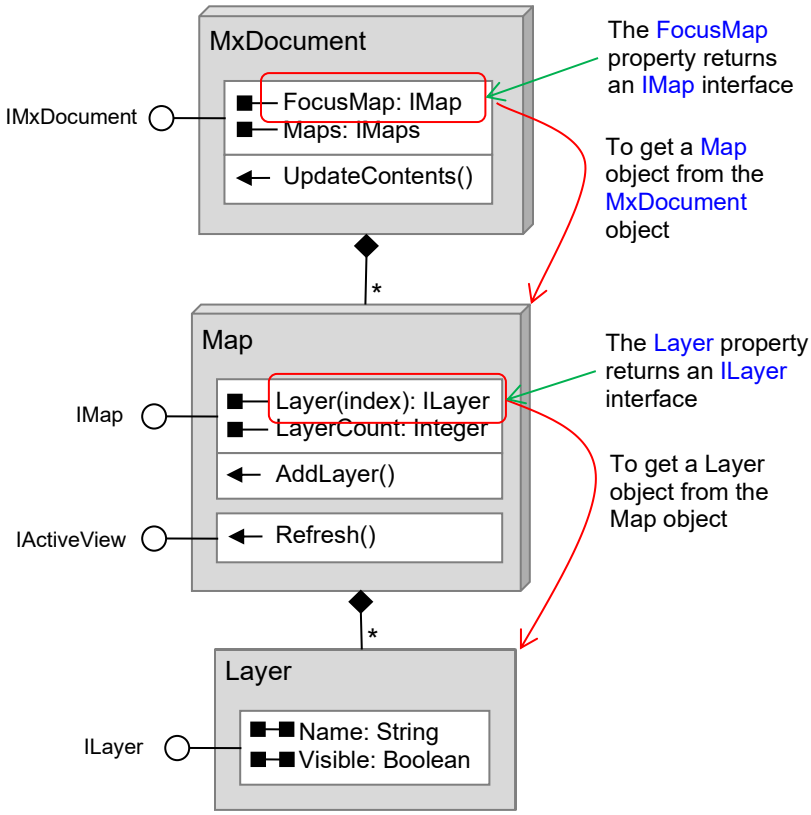


Figure 6 Hopping from one object to another with aggregation relationship

Figure 6 demonstrates that if two objects are related with an aggregation relationship, i.e. one object contains or is composed of the other, we can hop from the aggregate object to the constituent object. To make such hop, we need to find a property of the aggregate object that returns any interface of the constituent object. Since the relationship between the Map class and Layer class also satisfies these conditions, we can make a further hop from a map object to a layer object, i.e. to get a specific layer. For example,

```
Dim lyr As ESRI.ArcGIS.Carto.ILayer = map.Layer(0)
```

In the code above, we obtain the top layer in the map (data frame) by calling the Layer property by passing argument 0 to the property. The index of the top layer in a map is 0, second layer is 1, and so on so forth. The layer object with ILayer interface is assigned to a new variable *lyr* which is declared to point to the ILayer interface.

With the guide of the ArcObjects model diagrams provided by ESRI, we can virtually hop to any connected object by following relationships between components when necessary: any map layer, any feature, the geometry of any feature, any row in the attribute table, any attribute value, as well as any map symbol! Using the City of ArcObjects analogy, if we start from the airport (MxDocument), we can travel to any facility in the city by following the roads (relationships). What a fantastic City of ArcObjects!

3. Turning off of a layer

Once we arrive at a layer object by starting from the map document object, turning on/off the layer is straightforward – to set a True or False value to the Visible property on the ILayer interface (Figure 6). Please follow steps below to implement the *LayersOnOffButton* add-in button of the CityGIS project.

1) Open your CityGIS project. If you do not have the project, copy it from folder "VB2010" on the CD-ROM to your hard drive.
2) Double click on "LayersOnOffButton.vb" in the Solution Explorer to open the LayersOnOffButton class module.
3) Read through the following code and comments, and then enter the code into the OnClick() sub procedure.

```
Protected Overrides Sub OnClick()
     ' get the MxDocument object and assign it to new variable mxDoc
     Dim mxDoc As ESRI.ArcGIS.ArcMapUI.IMxDocument = My.ArcMap.Document

     ' get the focus map object from the MxDocument object
     ' and assign it to new variable map
     Dim map As ESRI.ArcGIS.Carto.IMap = mxDoc.FocusMap

     ' get the top layer (index 0) from the map object
     ' and assign it to new variable lyr
     Dim lyr As ESRI.ArcGIS.Carto.ILayer = map.Layer(0)

     ' set False to the Visible property of the layer object to turn it off
     lyr.Visible = False

     ' QI to the IActiveView interface of the map object
     Dim av As ESRI.ArcGIS.Carto.IActiveView = map
     av.Refresh()    ' refreshes map display

     ' call the UpdataContents method of the MxDocument object to update the TOC
     mxDoc.UpdateContents()
End Sub
```

4) Make sure there are no typos in your code and run your program.
5) When ArcMap opens, add a few layers into it. Bring up the CityGIS toolbar if it does not show.
6) Click the layers on/off button on the CityGIS toolbar. The top map layer should be turned off. If it does not work, check your code.

I am not going to further explain the code above since the comments in the procedure explain every step. Please read the comments carefully and make sure you understand what each line does. If your code works, that's great even if the button can only turns off the top layer at this point. You will gradually to improve functionality of this button.

4. Importing Namespaces

When you write code in ArcGIS add-in programming, typing long namespaces is tedious. Besides, interface names with long namespaces make the code look complicated. The good thing is, VB allows long namespaces to be omitted by using a statement to import necessary namespaces into a code module. An import statement should be placed above the class declaration and below all Option statements if any. For example,

```
Option Explicit On    ' this is optional, it is here only for demonstration

Imports ESRI.ArcGIS.ArcMapUI    ' import the ArcMapUI namespace
Imports ESRI.ArcGIS.Carto       ' import the Carto namespace

Public Class LayersOnOffButton  ' the import statements must be above this line
    ...
End Class
```

After importing a namespace, referring to the namespace is not needed when you use an interface or object. For example, once the ArcMapUI and Carto namespaces are imported in the LayersOnOffButton class module, the same code that turns off the top layer can be simplified as the following:

```
Protected Overrides Sub OnClick()
    Dim mxDoc As IMxDocument = My.ArcMap.Document
    Dim map As IMap = mxDoc.FocusMap
    Dim lyr As ILayer = map.Layer(0)

    lyr.Visible = False

    Dim av As IActiveView = map    ' QI
    av.Refresh()

    mxDoc.UpdateContents()
End Sub
```

Notice that the QI line is fine if statement "Option Strict On" is not present. Otherwise, you must explicitly cast the IMap interface to the IActiveView interface while performing query interface. For example,

```
Dim av As IActiveView = TryCast(map, IActiveView)
```

Please refer to Chapter 9 for more information about QI and interface casting.

5. Turning on/off all layers

Inside the OnClick event procedure of the LayersOnOffButton class, line 3 assigns layer 0 to variable *lyr*, and then line 4 turns that layer off. To turn off all layers in the current map, you can use a repetition to iterate over all layers and turn off each one. To do so, you can replace lines 3 and 4 by the following code:

```
Dim lyr As ILayer   ' declares a variable to point to the ILayer interface
' loop through all layers in the map
For i As Integer = 0 To map.LayerCount - 1
    lyr = map.Layer(i)      ' assigns the i's layer to variable lyr
    lyr.Visible = False     ' turns off the layer
Next
```

LayerCount is a property on the IMap interface of the Map object. It returns the number of layers in the map. Since layer indexes start from 0 (the top layer), the index of the last layer in the map is the total number of layers – 1, i.e. map.LayerCount – 1.

To turn on and off all layers by clicking the same button, you need to let the computer to remember the on/off state of layers or a previously performed action. This can be achieved by using a Boolean variable to preserve the on/off state of all layers. You may name this Boolean variable as *LayersOn*, and may set the initial value of this variable to True.

Where is the proper location to declare the *LayersOn* variable? Can it be declared in the button click event procedure? Please recall that variables have scopes and lifespan (Chapter 5). All (except Static) variables declared in a procedure are erased when the procedure finishes execution. If variable *LayerOn* is declared in the button click event, the variable will be erased after the event procedure is executed once. Therefore, the on/off state of layers is not remembered by the computer.

We can declare *LayersOn* as a class-level variable. A class-level variable stays alive as long as the instance of the class is alive. In the case of LayersOnOffButton class, all its class-level variables stay alive until the button is destroyed. When does that happen? It happens when the ArcMap application is closed. So a class-level variable in the LayersOnOffButton class can serve the purpose of preserving on/off state of layers. The LayersOn variable can be declared as the following:

```
Public Class LayersOnOffButton
    Inherits ESRI.ArcGIS.Desktop.AddIns.Button
    Private LayersOn As Boolean = True    ' this is a class-level variable that
                                          ' stays alive until ArcMap is closed
```

After declaring a class-level variable LayersOn, you can make the following change to the For … Next repetition block in the button click event procedure:

```
    ' reverse the value of variable LayersOn before the repetition
    LayersOn = Not LayersOn
    For i As Integer = 0 To map.LayerCount - 1
        lyr = map.Layer(i)          ' assigns the i's layer to variable lyr
        lyr.Visible = LayersOn      ' turns off the layer
    Next
```

After this change, start the add-in program and test the layers on/off button. The button should work like a charm.

6. Changing map scales

In the CityGIS project, the *MapScaleComboBox* add-in control is designed to allow users to zoom the map to a scale selected from a drop-down list. Three map scales have been initially added into the combo box. This section will instruct you to implement the zoom functionality.

Changing map scale by programming is not more complicated than changing map layer visibility. But there are a couple of key techniques you must know before implementing the functionality.

First, the IMap interface of the Map class has a two-way property named MapScale (open the "CartoObjectModel.pdf" object model diagram and zoom to the Map class) (Figure 7). The MapScale is a Double type property that allows you to set a map scale factor. For example, to set the map scale to 1:10,000, you need to set the scale factor 10000 to the MapScale property.

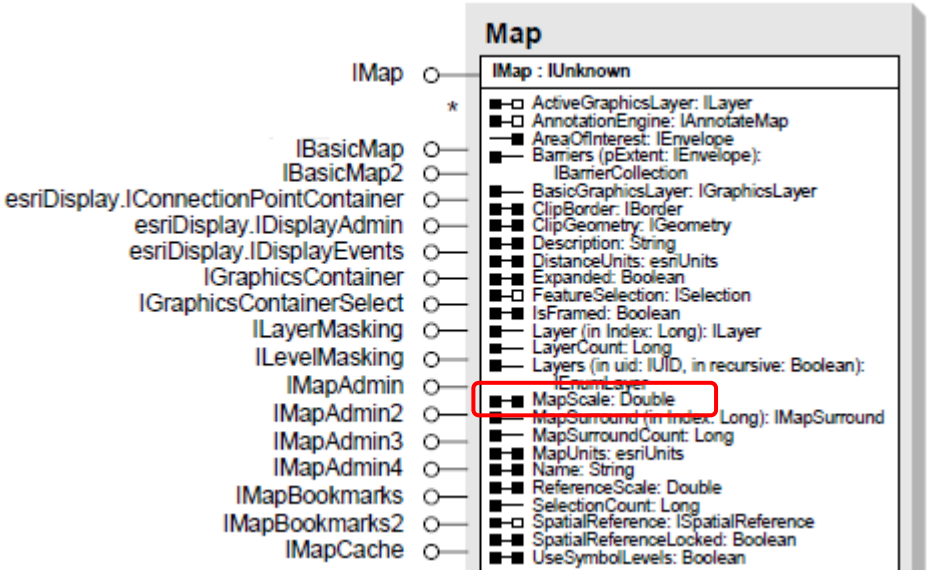


Figure 7 MapScale property on the IMap interface of the Map class

Second, you need to use the OnSelChange event of the add-in combo box. Unlike ordinary VB programming in which you can select an event from the event drop-down list in the code editor, you have to manually enter this event procedure framework for this ArcGIS add-in control. Fortunately, the IntelliSense of VB can help you accomplish the task. To add the OnSelChange event procedure into the MapScaleComboBox class, please type the following code inside the class. Make sure you do not type inside any existing procedure.

```
Protected Overrides Sub OnSelChange
```

When you see the words “OnSelChange(Integer)” highlighted in IntelliSense as you type, hit the Enter key to finish. A blank structure of the OnSelChange event procedure will be completed by IntelliSense as the following:

```
Protected Overrides Sub OnSelChange(cookie As Integer)
    MyBase.OnSelChange(cookie)
End Sub
```

This event procedure is fired whenever the selected item in the map scale combo box changes. Now, you are ready to add code into this event procedure to implement map scale functionality.

```
Protected Overrides Sub OnSelChange(cookie As Integer)
    ' declare a string variable to hold user selected map scale (e.g. "1:10,000")
    Dim strscale As String = Me.Value   ' Note: Me refers to the combo box class

    ' use SubString method to extract scale factor (e.g. "10,000")
    ' from the scale string (e.g. "1:10,000")
    strscale = strscale.Substring(2)  ' 2 is the starting index of scale factor

    ' convert the factor to a Double number and assign it to a new variable scale
    Dim scale As Double = CDbl(strscale)

    Dim mxDoc As IMxDocument = My.ArcMap.Document   ' gets the map document
    Dim map As IMap = mxDoc.FocusMap    ' gets the focus map

    map.MapScale = scale    ' assigns the scale factor to the MapScale property

    ' QI to the IActiveView interface of the map object
    Dim av As IActiveView = map
    av.Refresh()            ' refresh the map

    MyBase.OnSelChange(cookie)      ' leave this line here, do not touch it.
End Sub
```

Generally, this event procedure carries out the job in six steps without branches and loops. The procedure is as straightforward as making sandwiches:

1) get the user selected map scale (a string such as “1:10,000”) from the combo box;
2) extract the scale factor (e.g. “10,000”) from the map scale string;
3) convert a string scale factor to a Double type number;
4) get the focus map from the MxDocument object;
5) assign the scale value to the MapScale property;
6) get the IActiveView interface and call its Refresh() method to refresh the map.

You may load the Flagstaff parcel feature class (in folder Data on the CD-ROM) into ArcMap to test the map scale combo box.

7. Getting coordinates of mouse location

The CityGIS project has a tool (AddBusStopTool) intended for creating bus stops by clicking on the map. In order to implement this functionality, you must know the coordinates of the clicking points on the map.

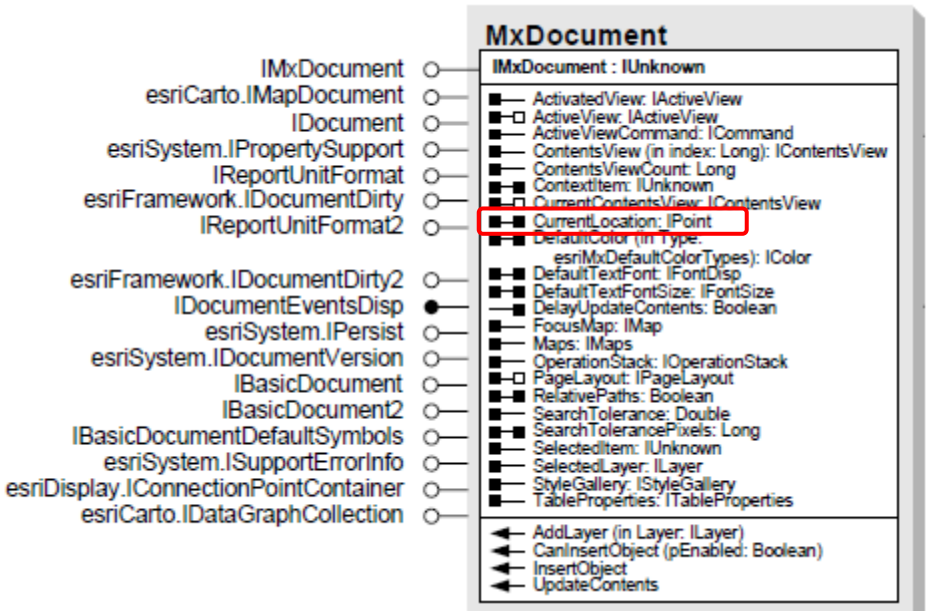


Figure 8 CurrentLocation property on the IMxDocument interface of the MxDocument class

Although displaying a bus stop graphic in the map is a little complicated (therefore it has to be accomplished a little later), to get the mouse location is quite simple. Figure 8 shows that the IMxDocument interface has a property named CurrentLocation which returns an IPoint interface, i.e. a point object with IPoint interface (Figures 9). So you can easily get the mouse location by calling the CurrentLocation property. You just need to declare a variable to hold the point object obtained from the CurrentLocation property in order to use the object in subsequent code. Since the Point class is included in the ESRI Geometry assembly, you must load the ESRI.ArcGIS.Geometry assembly into the CityGIS project as a reference in order to use the Point class and other ArcGIS geometry components. Please refer to Section 2 if you need help in loading an ESRI assembly into the project. After loading this assembly, check the solution Reference list to make sure the assembly is added to the project.

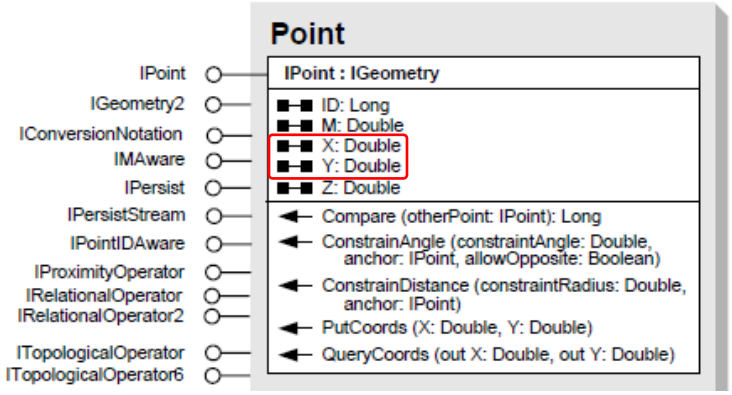


Figure 9 X and Y properties on the IPoint interface of the Point class

Now, let's open the AddBusStopTool class module and add the following import statements in the header of the class module (above the class declaration):

```
Imports ESRI.ArcGIS.Framework
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Geometry
```

Getting the mouse cursor location can be carried out in the OnMouseDown event procedure. Like the OnSelChange event of the combo box, you have to type the framework of the event procedure with the help of IntelliSense. Please type the following code in the AddBusStopTool class.

```
Protected Overrides Sub OnMouseDown
```

When you see `OnMouseDown(arg As ESRI.ArcGIS.Desktop.AddIns.Tool.MouseEventArgs)` highlighted in IntelliSense as you type, hit the Enter key to complete the event procedure structure. A blank OnMouseDown event procedure should look like to the following.

```
Protected Overrides Sub OnMouseDown(arg As ...)

    MyBase.OnMouseDown(arg)
End Sub
```

Then you can add four lines of code into the event procedure as the following:

```
Protected Overrides Sub OnMouseDown(...)
    ' get the MxDocument object
    Dim mxDoc As IMxDocument = My.ArcMap.Document
    ' get the mouse location from the CurrentLocation property and
    ' assign the derived point object to a new variable p (pointing to IPoint)
    Dim p As IPoint = mxDoc.CurrentLocation

    ' get the application object and assign it to a new variable app
    Dim app As IApplication = My.ArcMap.Application
    ' display coordinates in the first message panel on the status bar
    app.StatusBar.Message(0) = "Coordinates: x=" & p.X & ", y=" & p.Y

    MyBase.OnMouseDown(arg)     ' leave this line here, do not touch it
End Sub
```

The key of this procedure is the second line which calls the CurrentLocation property of the MxDocument object. The property returns a point object with the IPoint interface. In the same line, this object is assigned to new variable *p* declared to the IPoint interface.

The third line of code inside the procedure gets the Application object and assigns it to new variable app. Therefore, variable app is a proxy of the Application object. Finally, the x and y coordinates of the point object are displayed in the first message panel (Message(0)) on the status bar (StatusBar) of the ArcMap application (app). Please see Figure 10 and the complete model diagram document "FrameworkObjectModel.pdf" about accessing the status bar of ArcMap.

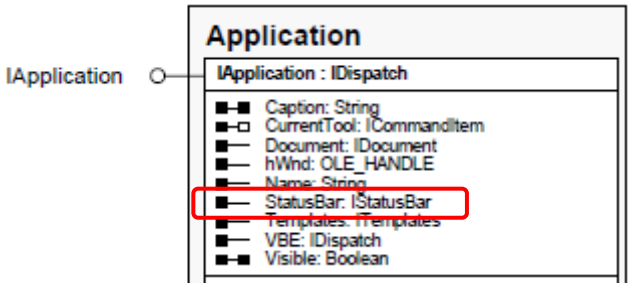


Figure 10 StatusBar property on the IApplication interface of the Application class

When you test the add-in project, you will find that the default mouse cursor of the bus stop tool is an arrow. You can set a preferred mouse cursor to the tool to distinguish it from other tools. For example, you may set a hair cross icon as the mouse cursor of the tool by adding the following line in the constructor (i.e. the New() sub procedure) of the AddBusStopTool class.

```
Me.Cursor = Windows.Forms.Cursors.Cross
```

Summary and Highlights

When an ArcGIS desktop add-in project is started, two objects, Application and MxDocument, are automatically instantiated and ready for use in programming. In an ArcMap add-in project, the Application object represents the ArcMap application represented by the ArcMap window, and the MxDocument object represents the map document (.mxd file). These two object are usually two starting points from which we can access other components of an application. If we compare the ArcObjects to a city, these two objects are two airports at which we arrive and from where we can start to explore the city.

In ArcObjects, when two objects are related with an aggregation relationship, we can hop from the aggregate component to the constituent component. This is illustrated by Figure 2 in which an application contains one map document, a map document can contain multiple maps, and a map can contain multiple layers. Therefore, we can hop from the document to a map, and from a map to a layer. The hopping is carried out by calling a property of the aggregate object that returns the constituent object.

A map object represents a data frame in an ArcMap document. A map document has an active data frame that can be accessed through the FocusMap property on the IMxDocument interface. A map object has a Layer property on the IMap interface. This property returns a layer of a specified index. The index of the top layer in a data frame is 0. The IMap interface has a LayerCount property that tells how many layers the map contains. It also has a MapScale property that can be used to change the map scale by setting a scale factor. A layer object has a Visible property on the ILayer interface that can be set to True or False to turn on or off the display of the layer. Components Map and Layer are included in the Carto assembly. To use these ArcObjects components, the Carto assembly must be loaded into the add-in project as a reference.

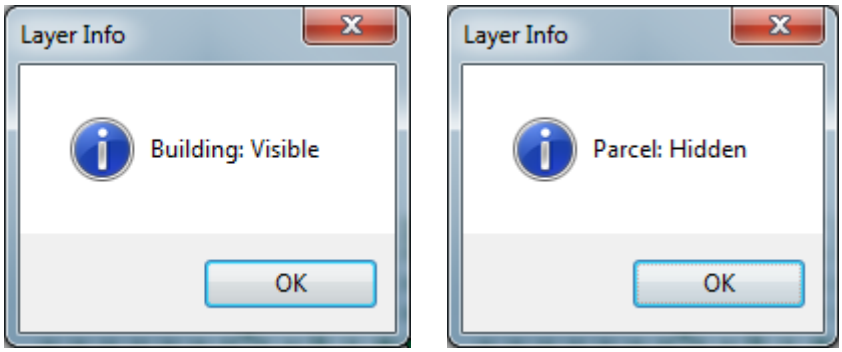
The map document object (MxDocument) has a CurrentLocation property on the IMxDocument interface. This property returns a point object with IPoint interface that indicates the mouse

cursor location. The point component is included in the Geometry assembly. In order to use the point object, the Geometry assembly must be loaded into the add-in project as a reference.

Exercises

1. Answer the following questions.
   1) What key objects are in the Framework assembly?
   2) What key objects are in the ArcMapUI assembly?
   3) What is the Application object?
   4) What is the MxDocument object?
   5) Can you name one property on the IApplication interface?
   6) Can you name one property on the IMxDocument interface?
   7) Can you name one method on the IMxDocument interface?
   8) How do you add additional assemblies into your projects?
   9) What is the Carto assembly?
   10) Please name two or more objects in the Carto assembly.
   11) Please name two or more properties on the IMap interface.
   12) What object does the IActiveView interface belong to?
   13) Please name one or more methods on the IActiveView interface.
   14) Please name one or more properties on the ILayer interface.
   15) How can you hop from one object to another in ArcObjects?
   16) What conditions are needed in order to hop from one object to another?
   17) Given that you have an object pointing to the ILayer interface, how do you turn the layer off?
   18) How do you refresh the map after making changes to it?
   19) How do you update the Table Of Contents (TOC) after making changes to a map?
   20) Why do you declare "Imports <namespace>" in the header of a code module?
   21) Given that you have a map object (named *map*) pointing to the IMap interface, how do you get the IActiveView interface?
   22) How do you find the number of layers in a map?
   23) What property or method do you use to change a map's scale?
   24) How do you bring up the OnSelChange event procedure of a combo box?
   25) How do you bring up the OnMouseDown event procedure of an add-in tool?
   26) What ArcObjects assembly contains the ESRI Point class?
   27) How do you get the mouse cursor location?
   28) What object does the CurrentLocation property on the IMap interface return?
   29) How do you change the mouse cursor icon?
   30) How do you write a message in the ArcMap status bar?

2. Create a new ArcMap add-in project named "Exercise9.2". The project should have a toolbar (caption: "Exercise 9.2 toolbar") that contains the following add-in components
   1) An add-in button named "GetLayerInfoButton". When the button is clicked it reports the name and visibility state of each layer in the current map by message boxes. The output of your program may look like the following screenshots.



2) An add-in tool named “CursorLocationTool”. When this tool is selected and the user moves the mouse cursor over the map, the x and y coordinates keeps changing in a panel on the status bar.
3) An add-in button named “CloseApplicationButton” which when clicked, pops up a message to ask the user for confirmation of closing the ArcMap window. If the user confirms closing, it closes ArcMap.